# DAY 5

## Functions & Scope

**PHASE 2 — BEGINNER**

| Session Length | Prerequisites | Format |
|---|---|---|
| 2 hours | Days 1–4 | Live + Hands-on |

30-Day Python Programming Course — Instructor Session Guide

# Session Overview

**Goal:** Introduce functions as reusable blocks of code — definitions, parameters, return values, default arguments, *args/**kwargs, and variable scope via the LEGB rule.

**Outcome:** By the end of this session, students can write their own functions with flexible arguments, understand what a function returns, and reason about where a variable lives (local, enclosing, global, or built-in).

## Session Timeline

| Time | Segment |
|---|---|
| 0:00 – 0:10 | Recap Day 4 (loops) + intro to functions |
| 0:10 – 0:35 | Defining functions, parameters, arguments, return values |
| 0:35 – 0:55 | Default arguments + guided practice |
| 0:55 – 1:05 | Break |
| 1:05 – 1:35 | *args and **kwargs |
| 1:35 – 1:55 | Scope and the LEGB rule |
| 1:55 – 2:00 | Wrap-up, mini-challenges, preview Day 6 |

**■ TIP**

Functions are the single biggest mindset shift so far in the course — students move from writing scripts top-to-bottom to designing reusable pieces. Spend extra time on the 'why', not just the syntax.

# Prerequisites & Preparation

## Requirements

| Requirement | Details |
|---|---|
| Python 3.12 | Installed and verified from Day 1 |
| VS Code | With Python extension active |
| Prior knowledge | Variables, control flow, loops (Days 2–4) |
| Working folder | day05/ folder created for today's files |

## Instructor Pre-Session Checklist

■ Review Day 4 loop exercises so recap references real student code

■ Open day05_01_functions_basics.py, day05_02_args_kwargs.py, and day05_03_scope_legb.py in VS Code, ready to live-code

■ Prepare 2–3 real-world function examples beyond the Mumbai grocery app in case students want more variety

■ Have the practice problem set ready to share at session end

# Step-by-Step Instruction Guide

## STEP 1 — Defining Functions with def

### What to Say:

*"Every time you write the same block of code more than once, that's a signal to wrap it in a function. Think of a function like a recipe — you write it once, and then you can 'cook' it (call it) as many times as you like."*

```
def greet_student():
    print("Namaste! Welcome to Day 5.")

greet_student()
# Output:
# Namaste! Welcome to Day 5.
```

**■ WATCH OUT**

Students confuse *defining* a function with *calling* it. Emphasize: writing def just teaches Python the recipe — nothing runs until you call greet_student().

## STEP 2 — Parameters, Arguments & Return Values

### What to Say:

*"Parameters are like blanks in the recipe — student_name, marks, price. Arguments are what you actually fill those blanks with when you call the function. And return is how the function hands a result back to you, instead of just printing it."*

```
def calculate_total(price, quantity):
    total = price * quantity
    return total

bill_amount = calculate_total(250, 3)
print(f"Total bill: Rs. {bill_amount}")
# Output:
# Total bill: Rs. 750
```

**■ DONE WHEN**

Students are ready to move on when they can explain, in their own words, the difference between print() inside a function and return.

## STEP 3 — Default Arguments

### What to Say:

*"Sometimes a parameter almost always has the same value. Instead of typing it every time, give it a default — like a train booking function that assumes Sleeper class unless told otherwise."*

```
def book_ticket(destination, travel_class="Sleeper"):
    print(f"Booking to {destination} in {travel_class}.")

book_ticket("Mumbai")
book_ticket("Delhi", "AC 3 Tier")
# Output:
# Booking to Mumbai in Sleeper.
# Booking to Delhi in AC 3 Tier.
```

| Error | Cause | Fix |
|---|---|---|
| SyntaxError: non-default argument follows default | Required parameter placed after a default one | Put all default parameters last in the signature |
| TypeError: missing required argument | Forgetting to pass a non-default parameter | Check the function signature and supply all required args |

## STEP 4 — *args and **kwargs

### What to Say:

*"What if you don't know how many arguments someone will pass? *args collects extra positional values into a tuple, and **kwargs collects extra keyword values into a dictionary. Think of a Mumbai grocery order — you never know how many items someone will order."*

```
def total_expenses(*amounts):
    return sum(amounts)

print(total_expenses(120, 340, 75))
# Output:
# 535

def student_profile(**details):
    for key, value in details.items():
        print(f"{key}: {value}")

student_profile(name="Sanjay", city="Mumbai")
```

**■ TIP**

OS note: no OS-specific commands are needed for this step — it's pure Python syntax, works identically on macOS / Windows / Linux.

## STEP 5 — Scope and the LEGB Rule

### What to Say:

*"Where does Python look for a variable's value? First inside the current function (Local), then any function it's nested inside (Enclosing), then the top of the file (Global), then Python's own built-ins (Built-in). This order is LEGB."*

```
total_registrations = 0

def register_student():
    global total_registrations
    total_registrations += 1

register_student()
print(total_registrations)
# Output:
# 1
```

**■ WATCH OUT**

The single most common bug at this stage: forgetting `global` and getting an UnboundLocalError. Walk through the error message line-by-line with students so they recognize it next time.

## Verification

**■ DONE WHEN**

Students are done with this step when they can correctly predict the output of a 3-level nested function (local / enclosing / global) before running it.

# Session Wrap-Up

## Student Checklist

■ I can define a function with def and call it

■ I understand the difference between parameters and arguments

■ I can write a function that returns a value

■ I can set default argument values

■ I can use *args and **kwargs in a function signature

■ I can explain the LEGB rule in my own words

## Key Concepts Recap

| Concept | One-Line Summary |
|---|---|
| def | Defines a reusable block of code |
| return | Sends a value back to the caller |
| Default arguments | Fallback value used when an argument is omitted |
| *args | Collects extra positional arguments into a tuple |
| **kwargs | Collects extra keyword arguments into a dictionary |
| LEGB | Local → Enclosing → Global → Built-in lookup order |

## Next Day Preview

**Day 6 — Data Structures:** Lists, tuples, dictionaries, sets, comprehensions, and slicing. Students will start storing and manipulating collections of data, building directly on the functions they wrote today.

## Troubleshooting Notes

■ **WATCH OUT**

If students consistently forget `return` and expect print() to 'send back' a value, have them write a function, call it, and try to print(result) — the None output makes the distinction click fast.

# Quick Reference Card

## Daily Workflow Commands

```
# Activate virtual environment
source .venv/bin/activate      # macOS/Linux
.venv\Scripts\activate         # Windows

# Run a Python file
python day05_01_functions_basics.py
```

## VS Code Shortcuts

| Shortcut | Action |
|---|---|
| Ctrl/Cmd + ` | Open integrated terminal |
| F5 | Run/debug current file |
| Ctrl/Cmd + / | Toggle line comment |
| Shift + Alt + F | Format document |

## Tool Summary — Day 5

| Tool / Keyword | Purpose |
|---|---|
| def | Define a function |
| return | Return a value from a function |
| param=default | Set a default argument value |
| *args / **kwargs | Accept variable positional/keyword arguments |
| global | Modify a global variable from inside a function |